

GPU Memory Optimization

GPU performance is often limited not by arithmetic, but by how efficiently data moves through the memory hierarchy. This interview guide covers the ten memory topics most often used to test whether a candidate can connect GPU hardware behavior to kernel performance.

Coalesced Memory Access

1. Overview

Definition. Coalesced access occurs when threads in a warp access nearby, properly aligned global-memory addresses that the GPU can serve with a small number of memory transactions.

It matters because global memory has high bandwidth but also high latency. A kernel can request the same number of useful bytes yet run much slower if those bytes are scattered across many cache lines or memory sectors. Coalescing is used in vector operations, image processing, matrix kernels, reductions, and nearly every bandwidth-bound CUDA kernel. Interviewers ask about it because it reveals whether you understand the difference between source-level loads and hardware memory transactions.

2. Core Idea

Imagine 32 people collecting books. If all books are on one shelf, one cart trip can fetch them; if each book is on a different shelf, many trips are needed. A warp behaves similarly.

```
// Usually coalesced: adjacent threads read adjacent floats.  
int i = blockIdx.x * blockDim.x + threadIdx.x;  
float x = input[i];  
  
// Usually poorly coalesced: adjacent threads are stride elements apart.  
float y = input[i * stride];
```

Step by step:

1. A warp issues a load instruction.
2. Each active thread supplies an address.
3. Hardware groups those addresses by aligned memory sectors/cache lines.
4. It issues enough transactions to cover all requested addresses.
5. Useful-byte efficiency is $\text{requested bytes} / \text{transferred bytes}$.

Modern GPUs coalesce dynamically; the old rule that threads must follow one exact pattern is too rigid. The durable rule is: minimize the number of aligned sectors touched by a warp.

3. Important Subtopics

Contiguous access

Thread `t` accesses element `base + t`. This usually gives the best transaction efficiency. Interview angle: map the fastest-changing data dimension to `threadIdx.x`.

Strided access

Thread `t` accesses `base + t * stride`. With a large stride, one warp may touch one sector per thread. This matters in column access to row-major matrices. Interviewers often ask how transpose tiling fixes it.

Alignment and transaction boundaries

Even contiguous addresses may span an extra boundary if the first address is misaligned. Example: a warp reading 32 floats starting at an offset of one float can touch more sectors than a naturally aligned read.

Structures of arrays (SoA)

```
struct ParticleAoS { float x, y, z, mass; };  
// SoA: float* x; float* y; float* z; float* mass;
```

If every thread needs only `x`, SoA places all requested values together; AoS interleaves unused fields. Interview angle: AoS may still be reasonable when every thread consumes the entire object.

Partial warps and divergence

Only active lanes generate addresses. Boundary warps may have lower utilization, but correctness comes first. Predicated bounds checks are normally preferable to unsafe out-of-range loads.

4. Real-World Example

In an image-brightness kernel, assign adjacent pixels to adjacent threads. A warp then reads a contiguous run of pixels, modifies them, and writes a contiguous run. Assigning each lane to a different row creates a pitch-sized stride and many transactions. Image frameworks therefore design layouts, pitches, and thread mappings together.

5. Diagrams / Mental Models

```
Coalesced (one compact region)  
lanes:      0  1  2  3  4  5 ...  
elements: [0][1][2][3][4][5]...  
sectors:  |----- few aligned sectors -----|  
  
Strided (many regions)  
lane 0 -> [0]      lane 1 -> [16]    lane 2 -> [32] ...  
          sector A      sector B      sector C
```

Pattern	Warp addresses	Typical result
Unit stride	Consecutive	Few transactions

Pattern	Warp addresses	Typical result
Small stride	Regular but gapped	More sectors and wasted bytes
Large stride	Far apart	Potentially one transaction per lane
Broadcast	Same address	Often served efficiently through cache/broadcast behavior

6. Common Interview Questions

1. **What is coalescing?** Combining a warp's nearby global-memory requests into few transactions. **Expected:** warp-level reasoning, alignment, useful bytes. **Mistake:** saying calls are merged by the compiler.
2. **Why does it improve performance?** It raises transaction efficiency and effective bandwidth. **Expected:** fewer sectors, not lower algorithmic byte count. **Mistake:** claiming it removes global-memory latency.
3. **Are consecutive thread IDs enough?** No; their *addresses* must be nearby. **Expected:** distinguish execution mapping from data mapping. **Mistake:** inspecting indices but not the address expression.
4. **How does stride affect access?** Larger stride spreads addresses over more sectors. **Expected:** explain wasted transferred bytes. **Mistake:** calling all strided access uncoalesced without considering stride and element size.
5. **Why can matrix column access be slow?** Row-major columns are separated by row width. **Expected:** transpose/tile suggestion. **Mistake:** blaming arithmetic.
6. **AoS or SoA?** SoA favors coalescing when threads read the same field; AoS may suit whole-object consumption. **Expected:** workload-dependent answer. **Mistake:** declaring SoA universally superior.
7. **Does a cache fix poor coalescing?** It may soften repeated loads, but cold scattered accesses still generate many sectors. **Expected:** coalescing and caching are related but distinct. **Mistake:** treating cache hits as guaranteed.
8. **What metric would you inspect?** Requested-versus-actual global load/store throughput, sectors per request, and memory throughput in Nsight Compute. **Expected:** profile rather than guess. **Mistake:** using occupancy alone.
9. **Can writes be coalesced?** Yes; adjacent lane stores should target nearby aligned addresses. **Expected:** mention write transactions. **Mistake:** discussing reads only.
10. **How do boundary checks affect coalescing?** The last warp may have inactive lanes but active contiguous lanes remain compact. **Expected:** correctness and modest tail inefficiency. **Mistake:** removing bounds checks for speed.

7. Deep-Dive Questions

1. **Why is "one transaction per warp" an oversimplification?** Transaction sizes, sectors, access width, alignment, architecture, and cache state differ; count the memory regions touched.
2. **Can vector loads help?** `float4` can reduce instruction count when naturally aligned, but it does not rescue a bad per-thread layout and may increase register demand.
3. **How would you optimize a transpose?** Read rows coalesced into a shared-memory tile, synchronize, then write columns as coalesced rows in the transposed coordinate system.
4. **What happens with ECC?** Extra transfer overhead can magnify inefficient scattered writes, but exact costs are architecture-dependent.
5. **How do pitched allocations matter?** Pitch aligns rows for efficient access; index with the returned pitch rather than assuming `width * sizeof(T)`.

8. Comparison Tables

Coalesced access	Scattered access
Few sectors per warp	Many sectors per warp
High useful-byte ratio	Bandwidth wasted on unused bytes
Usually contiguous/nearby addresses	Large stride or irregular addresses
Best for bulk array processing	Sometimes unavoidable in graphs/sparse data

9. Common Mistakes

- Optimizing thread-block shape without checking the resulting address expression.
- Assuming aligned allocation guarantees every offset is aligned.
- Confusing coalescing with cache locality.
- Using AoS when only one field is consumed.
- Ignoring stores, element width, pitch, and boundary warps.

10. Edge Cases / Special Cases

Broadcasts, repeated addresses, cached reads, sparse gathers, and atomics do not fit the simple consecutive-address model. Local-memory spills are also global-memory traffic, although their lane-based layout is designed to coalesce common spill patterns. Architecture-specific transaction details change, so optimize the measured sector count rather than memorizing one generation's rules.

11. How to Explain in Interview

"Coalescing means arranging a warp's global-memory addresses so they fall into as few aligned memory sectors as possible. I normally map adjacent `threadIdx.x` lanes to adjacent elements, check alignment and layout, then verify transaction efficiency in a profiler."

12. Quick Revision Notes

- Unit-stride warp access is the usual goal.
- Address pattern matters, not merely thread numbering.
- SoA often beats AoS for field-wise kernels.
- Misalignment and stride increase sectors touched.
- Trap: caches may mask, but do not redefine, poor coalescing.

13. Practice Tasks

1. Write vector-add kernels using unit stride and configurable stride; compare effective bandwidth.
2. Implement naive and tiled matrix transpose.
3. Convert a particle update from AoS to SoA and profile requested/actual bytes.
4. Draw the addresses touched by one warp for strides 1, 2, 8, and 32.

14. Final Cheat Sheet

Item	Answer
Core definition	Few aligned transactions serve a warp's addresses
Why it matters	Preserves usable global-memory bandwidth
Most asked	Stride, transpose, AoS vs SoA, alignment
Main comparison	Compact addresses vs scattered addresses
One-line answer	"Adjacent lanes should usually access adjacent, aligned data."

Shared-Memory Tiling

1. Overview

Definition. Shared-memory tiling divides data into blocks that cooperating threads load from global memory into fast on-chip shared memory and reuse.

It matters when multiple operations reuse the same values. Tiling cuts redundant global loads and can transform access patterns, as in matrix transpose. It is used in GEMM, convolution, stencils, reductions, histograms, and image filters. Interviewers use it to test synchronization, indexing, reuse, and resource trade-offs.

2. Core Idea

Shared memory is a team workbench for one thread block. Instead of every worker repeatedly walking to a warehouse, the team brings a tile to the bench once.

For matrix multiplication $C = A \times B$, a thread computing one $C[\text{row}, \text{col}]$ needs a row of A and a column of B . A tile-based kernel:

1. Assigns a block to a tile of C .
2. Coalescently loads one tile of A and one of B into shared memory.
3. Calls `__syncthreads()` so every load is visible.
4. Accumulates products using the on-chip tiles.
5. Synchronizes before overwriting shared storage with the next tiles.
6. Writes the result once.

```
__shared__ float As[T][T], Bs[T][T];
As[ty][tx] = A[row * n + tileStart + tx];
Bs[ty][tx] = B[(tileStart + ty) * n + col];
__syncthreads();
for (int k = 0; k < T; ++k) sum += As[ty][k] * Bs[k][tx];
__syncthreads();
```

3. Important Subtopics

Data reuse

Tiling pays when loaded values are reused. In $T \times T$ matrix multiplication, each tile value can serve roughly T computations. Interview angle: arithmetic intensity rises because global bytes fall.

Cooperative loading

Threads should divide tile loads evenly and use coalesced accesses. One thread loading an entire tile serializes work.

Synchronization

`__syncthreads()` is a block-wide barrier and memory-ordering point for shared memory. All non-exited threads in the block must reach it consistently; placing it in divergent control flow can deadlock or be invalid.

Halo regions

Stencil and convolution tiles often include neighboring "halo" cells. The tile is larger than the output region because boundary outputs need adjacent inputs.

Tile-size trade-offs

Large tiles improve reuse but consume more shared memory and registers, potentially lowering occupancy. The best tile fits the algorithm and hardware; it is measured, not assumed.

Static and dynamic shared memory

Static shared memory has compile-time size. `extern __shared__` is sized at launch and suits runtime tile/radius sizes. Interviewers expect correct byte-count calculation.

4. Real-World Example

A 2D blur reads each pixel and its neighbors. Without tiling, overlapping windows reload the same pixels many times. With tiling, a block loads its output area plus halo once; threads reuse those pixels for many output calculations. Boundary threads clamp, pad, or conditionally load out-of-image values.

5. Diagrams / Mental Models

```

Global memory          One thread block
large image    --cooperative--> [ halo | output tile | halo ] shared
                                | many reuses |
                                v
                                output tile to global

```

Question	Tiling benefit
Same value read by many threads?	Load once, reuse in shared memory
Global layout awkward for output?	Stage and rearrange data

Question	Tiling benefit
No reuse and already coalesced?	Shared memory may add overhead only

6. Common Interview Questions

1. **What is tiling?** Processing a block-sized data subset through shared memory. **Expected:** cooperative load, reuse, synchronization. **Mistake:** describing ordinary blocking without the memory movement.
2. **Why is shared memory faster?** It is on-chip and explicitly managed, with much lower latency than uncached global memory. **Expected:** qualify bank conflicts. **Mistake:** saying it is always single-cycle.
3. **When does tiling help?** When data is reused or rearranged for coalescing. **Expected:** reuse factor. **Mistake:** tiling a one-use streaming kernel.
4. **Why two barriers per iteration?** One protects reads after load; one prevents overwriting before all threads finish using the tile. **Expected:** identify both hazards. **Mistake:** keeping only the first barrier.
5. **How choose tile size?** Balance reuse, shared memory, registers, occupancy, coalescing, and bank behavior. **Expected:** benchmark candidates. **Mistake:** always choosing 32×32 .
6. **What is a halo?** Neighboring input cells required around an output tile. **Expected:** boundary handling. **Mistake:** omitting halo bytes from shared-memory sizing.
7. **Can blocks share shared memory?** Traditionally no; it is block-scoped. Newer cluster features are architecture-specific. **Expected:** standard programming model first. **Mistake:** using it for grid-wide communication.
8. **What if dimensions are not multiples of tile size?** Guard global loads/stores and initialize out-of-range tile entries safely. **Expected:** barriers remain uniformly reached. **Mistake:** returning before a required barrier.
9. **Does tiling always increase occupancy?** No; it consumes per-block shared memory and may reduce resident blocks. **Expected:** performance can improve despite lower occupancy. **Mistake:** optimizing occupancy as the final goal.
10. **How does tiling improve transpose?** Read and write global memory in coalesced directions, using shared memory to exchange coordinates. **Expected:** padding to avoid bank conflicts. **Mistake:** merely copying through shared memory.

7. Deep-Dive Questions

1. **How is arithmetic intensity changed?** Reusing each global load for multiple operations increases operations per global byte.
2. **Can warp shuffles replace shared memory?** For warp-local exchange, often yes; shuffles avoid block barriers but cannot directly communicate across warps.
3. **What is double buffering?** One tile is computed while another is loaded, sometimes with asynchronous copies; it hides latency but increases storage and complexity.
4. **Why can a smaller tile win?** It may allow more resident blocks, reduce register pressure, and handle boundaries more efficiently.
5. **What does an async copy pipeline change?** It overlaps global-to-shared movement with compute and can avoid intermediate registers, but requires staged synchronization and supported hardware.

8. Comparison Tables

Direct global access	Shared-memory tiling
Minimal code and barriers	Cooperative load plus barriers
Best for one-pass streaming	Best for reuse/rearrangement
More repeated global loads	Fewer global loads
No shared-memory footprint	May reduce occupancy

9. Common Mistakes

- Tiling without measurable reuse.
- Missing the barrier before reuse or overwrite.
- Returning from some threads before a block-wide barrier.
- Forgetting halo and partial-tile handling.
- Selecting a tile only from warp size.
- Fixing coalescing while introducing shared-memory bank conflicts.

10. Edge Cases / Special Cases

For very small working sets, hardware caches may already capture reuse. Constant memory may be better for uniform read-only coefficients. Register tiling may complement shared tiling. Shared-memory capacity and configurable L1/shared partitioning vary by GPU, and kernels using dynamic shared memory may require launch attributes for large allocations.

11. How to Explain in Interview

“Shared-memory tiling lets a block cooperatively load a reusable data tile once, synchronize, and perform many operations from on-chip memory. I use it when reuse or data rearrangement repays the load, barrier, and shared-memory cost.”

12. Quick Revision Notes

- Shared memory is block-scoped and explicitly managed.
- Reuse and coalescing transformation are the main benefits.
- Guard edges without making barriers divergent.
- Tile size trades reuse against resource occupancy.
- Trap: shared memory is not automatically faster if each value is used once.

13. Practice Tasks

1. Implement naive and tiled matrix multiplication.
2. Add partial-tile handling for arbitrary matrix sizes.
3. Implement tiled transpose with and without padding.
4. Build a 2D stencil tile with a one-cell halo.
5. Compare 8×8 , 16×16 , and 32×32 tiles using a profiler.

14. Final Cheat Sheet

Item	Answer
Core definition	Stage block-sized reusable data in shared memory
Why it matters	Fewer global loads and better access patterns
Most asked	Barriers, tile size, halos, transpose
Main comparison	Direct streaming vs cooperative reuse
One-line answer	"Load once per block, reuse many times, synchronize safely."

Bank Conflicts

1. Overview

Definition. Shared memory is split into banks that can serve parallel accesses. A bank conflict occurs when lanes in a warp request different addresses in the same bank during one instruction, forcing serialization.

Bank conflicts matter because they turn an apparently fast shared-memory operation into multiple service rounds. They occur in transposes, histograms, scans, matrix kernels, and irregular shared-memory tables. Interviewers ask about them to test whether you understand shared memory as banked hardware rather than a uniformly fast array.

2. Core Idea

Think of shared memory as several checkout lines. Thirty-two customers finish quickly if they choose different lines. If several need different items handled by the same line, they queue. If all ask for the exact same item, hardware can broadcast it.

For common 32-bank organizations and 32-bit words, a simplified mapping is:

```
bank = (byte_address / 4) mod 32
```

Thus `s[threadIdx.x]` maps consecutive lanes to consecutive banks. But `s[threadIdx.x * 32]` maps many lanes to bank 0. Exact bank width and supported multicast behavior are architecture-dependent.

3. Important Subtopics

Bank mapping

The bank is determined by address, not thread ID. Element size changes how addresses span banks. Interview angle: calculate bank indices for a warp.

Conflict degree

A two-way conflict needs roughly two serialized bank services; a higher-degree conflict costs more. The actual performance impact depends on instruction mix and architecture.

Broadcast and multicast

Multiple lanes reading the same address can generally be broadcast and is not treated like different-address contention. Interview trap: "same bank" alone is not enough to prove a conflict.

Padding

```
__shared__ float tile[32][33];
```

Adding one column changes the stride so column-wise access rotates across banks instead of repeatedly selecting the same bank. This is the classic tiled-transpose fix.

Atomics versus bank conflicts

Shared-memory atomics contend when threads update the same location; bank conflicts concern different addresses mapping to one bank. Both serialize, but for different reasons.

4. Real-World Example

In matrix transpose, threads load `tile[ty][tx]` row-wise, then read `tile[tx][ty]` column-wise. With width 32, column elements may map to the same bank, causing a severe conflict. Declaring `tile[32][33]` changes each row's starting bank and makes column reads distribute across banks.

5. Diagrams / Mental Models

```
No conflict: lane 0->bank 0, lane 1->bank 1, ... lane 31->bank 31
Conflict:    lane 0->bank 0, lane 1->bank 0, ... different addresses queue
Broadcast:   lanes 0..31 -> exactly the same address -> one value broadcast
```

Access	Likely behavior
<code>s[lane]</code> with 32-bit values	Conflict-free
<code>s[lane * 2]</code>	Repeated-bank pattern, often two-way
<code>s[lane * 32]</code>	Severe conflict
Every lane reads <code>s[0]</code>	Broadcast

6. Common Interview Questions

1. **What is a bank conflict?** Different addresses requested from the same shared-memory bank by one warp instruction. **Expected:** serialization. **Mistake:** calling any shared-memory race a conflict.
2. **Why are there banks?** To provide parallel bandwidth through independent partitions. **Expected:** address-based mapping. **Mistake:** describing cache sets.
3. **Are same-address reads a conflict?** Usually no; broadcast/multicast serves them efficiently. **Expected:** distinguish same and different addresses. **Mistake:** "same bank always conflicts."

4. **How does padding help?** It changes row stride and therefore bank mapping. **Expected:** 32×33 transpose example. **Mistake:** saying padding aligns data without explaining mapping.
5. **Does coalescing apply to shared memory?** Coalescing is mainly a global-memory transaction concept; shared memory is analyzed through banks. **Expected:** separate the mechanisms. **Mistake:** using the terms interchangeably.
6. **How do you detect conflicts?** Profile shared-memory wavefronts/conflict metrics and inspect address patterns. **Expected:** profiler plus calculation. **Mistake:** inferring solely from runtime.
7. **Can writes conflict?** Yes, different same-bank addresses can serialize. **Expected:** races are a separate correctness issue. **Mistake:** discussing reads only.
8. **Does a conflict produce wrong output?** No by itself; it is a performance issue. Data races may produce wrong output. **Expected:** correctness/performance distinction. **Mistake:** conflating serialization with races.
9. **How does element size matter?** Wider elements may span or use multiple bank resources and change mapping. **Expected:** architecture-dependent nuance. **Mistake:** blindly applying the 32-bit formula.
10. **Should every conflict be removed?** Only if material; padding or rearrangement has costs. **Expected:** measure. **Mistake:** optimizing a negligible metric at the expense of occupancy.

7. Deep-Dive Questions

1. **Why does [32][33] work?** A column stride becomes 33 words, so successive rows advance one bank modulo 32.
2. **How do vector types affect conflicts?** One lane's wide access may be decomposed across banks/transactions; analyze generated instructions and architecture metrics.
3. **Can warp shuffles eliminate conflicts?** For warp-local rearrangement, yes, by exchanging registers without shared memory.
4. **What about shared-memory atomics in histograms?** Privatizing bins per warp or adding replicated bins can reduce same-address contention and may alter bank pressure.
5. **Why might padding hurt?** It consumes more shared memory, changes alignment, and can reduce resident blocks; use it where it fixes a measured pattern.

8. Comparison Tables

Bank conflict	Data race
Performance problem	Correctness problem
Different addresses in same bank	Unsynchronized conflicting accesses to same location
Hardware serializes service	Result/order may be undefined
Fix layout/padding	Fix synchronization/atomicity

9. Common Mistakes

- Assuming shared memory has uniform access cost.
- Forgetting same-address broadcast.
- Applying one bank formula to every architecture and data width.
- Confusing bank conflicts, atomics, and data races.
- Adding padding without checking shared-memory occupancy.

10. Edge Cases / Special Cases

Bank count, width, dual-port behavior, instruction decomposition, and multicast capabilities vary. Compiler transformations may split a source-level vector access. A profiler reports executed behavior, which is more reliable than reasoning from source alone. Padding a multidimensional array changes its indexing stride but should not change logical bounds.

11. How to Explain in Interview

“Shared memory is divided into banks. If a warp accesses different addresses that map to the same bank, the instruction needs multiple service rounds. I fix systematic conflicts by changing the layout—often one element of padding—and confirm with shared-memory conflict metrics.”

12. Quick Revision Notes

- Address determines the bank.
- Different same-bank addresses serialize.
- Same-address reads may broadcast.
- Padding changes stride modulo bank count.
- Trap: bank conflict is not a data race.

13. Practice Tasks

1. Compute bank mappings for strides 1, 2, 16, 32, and 33.
2. Benchmark transpose tiles `[32][32]` and `[32][33]`.
3. Replace a warp-local shared-memory exchange with `__shfl_sync`.
4. Profile a shared histogram with and without bin privatization.

14. Final Cheat Sheet

Item	Answer
Core definition	Different addresses contend for one shared-memory bank
Why it matters	Conflicts serialize warp memory service
Most asked	Mapping, padding, broadcast, transpose
Main comparison	Bank conflict vs data race
One-line answer	“Change the address stride so lanes spread across banks.”

Register Usage

1. Overview

Definition. Registers are the fastest per-thread storage on a GPU. The compiler places local scalars, loop state, addresses, and intermediate values in registers when possible.

Registers matter because they enable fast computation, but the register file is finite and shared among resident warps. High registers per thread can reduce occupancy; excessive live values can spill into local memory, which resides in device memory. Registers are central to matrix math, reductions, instruction-level parallelism, and every compiled kernel. Interviewers ask about them to test resource trade-offs rather than the simplistic “more occupancy is always better” rule.

2. Core Idea

Registers are like each worker's pockets: immediate access, but limited. Giving each worker more tools reduces trips to storage, yet fewer workers fit in the workshop.

Step by step:

1. Compiler analyzes live values.
2. It assigns register numbers to values whose lifetimes overlap.
3. Registers per thread $\times$ threads per block determines much of a block's register allocation.
4. The SM admits only as many blocks/warps as its register capacity and other limits allow.
5. If allocation is insufficient, some values spill to local memory.

Example: unrolling a loop may expose more parallel arithmetic, but it also keeps more accumulators live and can raise register count.

3. Important Subtopics

Register pressure

Many simultaneously live values create pressure. Complex expressions, aggressive unrolling, large per-thread arrays, and many accumulators are common causes. Interview angle: shorten live ranges or reduce unrolling before blindly forcing a compiler cap.

Occupancy

Occupancy is resident active warps divided by the hardware maximum. Registers, shared memory, threads, and block limits constrain it. Enough occupancy helps hide latency; maximum occupancy is not necessarily maximum performance.

Spilling and local memory

“Local” means private address space, not on-chip location. Spilled registers and dynamically indexed thread-local arrays commonly use local memory, which is backed by device memory and cached.

Register reuse and instruction-level parallelism

Multiple independent accumulators allow the scheduler to overlap instruction latency. Reducing registers too far may destroy this parallelism.

Launch bounds and register limits

Compiler flags such as `--maxrregcount` and CUDA launch-bound annotations can influence allocation, but may cause spills. They are tuning tools, not automatic optimizations.

4. Real-World Example

A GEMM microkernel keeps several output values in per-thread registers, reusing loaded operands across fused multiply-add instructions. Increasing the register tile improves reuse and instruction-level parallelism until register pressure lowers residency or spills. Production libraries tune this balance for each architecture and matrix shape.

5. Diagrams / Mental Models

```
Per-SM register file (finite)
  registers/thread rises
    |
    +--> fewer resident threads/warps
    +--> possibly better per-thread reuse/ILP
    +--> if too high or forced too low: spills
```

Situation	Likely effect
Too few useful live values	More reloads/recomputation
Moderate register use	Fast temporaries and good latency hiding
Very high register use	Lower occupancy
Register spilling	Local-memory load/store traffic

6. Common Interview Questions

1. **What are GPU registers?** Fast per-thread storage allocated by the compiler. **Expected:** finite per-SM resource. **Mistake:** treating them as programmer-indexed shared storage.
2. **What is register pressure?** Demand from simultaneously live values. **Expected:** connection to allocation, occupancy, spills. **Mistake:** equating it only with variable count.
3. **What is spilling?** Values that do not remain in registers are stored in local memory. **Expected:** device-memory-backed cost. **Mistake:** claiming local memory is on-chip.
4. **How do registers affect occupancy?** Per-thread allocation can limit resident blocks/warps. **Expected:** allocation granularity and other limits. **Mistake:** assuming a smooth one-to-one curve.
5. **Is higher occupancy always faster?** No; once latency is sufficiently hidden, more occupancy may not help, while fewer registers can hurt ILP or cause spills. **Expected:** profile-based trade-off. **Mistake:** targeting 100% universally.
6. **Can shared memory replace registers?** It can hold block-shared or indexed data but has higher latency and bank behavior. **Expected:** different scope/use. **Mistake:** using shared memory as a free register extension.
7. **What code raises register usage?** Unrolling, many accumulators, long live ranges, inlining, large local objects. **Expected:** compiler decides final allocation. **Mistake:** counting source variables.
8. **How do you inspect usage?** Compiler resource reports, disassembly when needed, and profiler occupancy/spill metrics. **Expected:** check local loads/stores. **Mistake:** relying on source intuition alone.
9. **Should you set max register count?** Only after measurement; a lower cap may increase occupancy but trigger spills. **Expected:** benchmark. **Mistake:** setting it globally by habit.

10. **Why use multiple accumulators?** They create independent work that hides arithmetic latency.

Expected: ILP versus pressure. **Mistake:** calling them redundant.

7. Deep-Dive Questions

1. **Why can removing a variable not reduce registers?** The compiler may already coalesce lifetimes or optimize it away; live ranges and generated code matter.
2. **How does block size interact with registers?** Allocation occurs with hardware granularity; a small per-thread increase can cross a threshold and reduce resident blocks.
3. **Why can recomputation beat spilling?** A few arithmetic instructions may be cheaper than device-memory-backed local loads/stores.
4. **How do function calls affect registers?** Inlining may enlarge live ranges; non-inlined calls can require stack/local-memory state depending on compilation.
5. **Can local-memory accesses coalesce?** Common per-thread spill layouts are arranged to support coalescing across lanes, but they still add instructions and memory-hierarchy traffic.

8. Comparison Tables

Registers	Shared memory	Local memory
Per thread	Per block	Per thread address space
Fastest	Fast on-chip, banked	Backed by device memory, cached
Compiler allocated	Programmer managed	Often spills/arrays
Can limit occupancy	Can limit occupancy	Adds memory traffic

9. Common Mistakes

- Treating every local variable as one physical register.
- Forcing a register cap before checking spills.
- Assuming local memory is physically near the core.
- Chasing maximum occupancy instead of runtime.
- Excessively unrolling loops without measuring register growth.

10. Edge Cases / Special Cases

Register allocation is architecture- and compiler-dependent and occurs in allocation units, so thresholds can be discontinuous. Debug builds and device debugging may alter optimization and register counts. Predication can keep values live across branches. Per-thread arrays with compile-time constant indices may remain in registers; dynamic indexing often prevents scalarization.

11. How to Explain in Interview

"Registers are the GPU's fastest per-thread storage, but they are a finite SM resource. I want enough registers for reuse and instruction-level parallelism without reducing useful residency or spilling into local memory; I verify that balance with compiler and profiler data."

12. Quick Revision Notes

- Registers are per-thread and compiler allocated.
- Pressure depends on live ranges, not just variable declarations.
- Spills become local-memory traffic.
- Occupancy is a means to hide latency, not the goal.
- Trap: lowering register count can make performance worse.

13. Practice Tasks

1. Compile a kernel with different unroll factors and record registers/thread.
2. Use an occupancy calculator to find register-limited residency.
3. Force a low register cap and inspect spill loads/stores.
4. Compare one accumulator with four independent accumulators.

14. Final Cheat Sheet

Item	Answer
Core definition	Fast compiler-managed per-thread storage
Why it matters	Controls reuse, ILP, occupancy, and spills
Most asked	Pressure, occupancy, local-memory spills
Main comparison	Registers vs shared vs local memory
One-line answer	"Use enough registers to compute efficiently, but watch residency and spills."

Memory Alignment

1. Overview

Definition. An address is aligned to **N** bytes when it is a multiple of **N**. GPU allocations are suitably aligned at their base, but offsets, structure layouts, subarrays, and vector types can create misaligned accesses.

Alignment matters because hardware memory instructions and transactions operate on naturally aligned regions. Misalignment may require extra transactions, prevent vector instructions, or violate type requirements. It appears in buffers, pitched images, tensor layouts, structures, and host-device transfer APIs. Interviewers ask about it because alignment connects C/C++ layout rules with GPU transaction efficiency and correctness.

2. Core Idea

Imagine moving furniture through doorways. A sofa lined up with the doorway passes once; one starting across a divider may require two maneuvers.

```
16-byte aligned:  | [----- 16-byte value -----] |
misaligned:      |part 1| [----- value -----] |part 2|
```

Step by step:

1. Allocation returns an aligned base pointer.
2. Code adds an element or byte offset.
3. The final address—not merely the base—is checked against the access width.
4. A warp's range is divided into aligned sectors.
5. Misaligned or boundary-crossing accesses may touch extra sectors or compile into narrower instructions.

3. Important Subtopics

Natural alignment

A type is naturally aligned when its address meets its alignment requirement. `float` commonly needs 4-byte alignment; vector and aggregate requirements differ. Interview angle: `alignof(T)` and `offsetof` matter more than assumptions.

Allocation alignment versus offset alignment

`cudaMalloc` provides a well-aligned base, but `reinterpret_cast<float4*>(p + 1)` can still be misaligned. Always reason about the final pointer.

Structure padding

Compilers insert padding to align members. This changes size and offsets and can affect AoS bandwidth. Host and device must agree on layout when sharing raw structs.

Vectorized loads/stores

Types such as `float2/float4` can encourage wider instructions when aligned and useful. They reduce instruction count, not necessarily bytes transferred, and may raise register use.

Pitch and row alignment

Pitched allocation adds row padding so each row begins at a favorable boundary. The returned pitch is measured in bytes and must be used in address calculation.

4. Real-World Example

A video frame may have a logical row width of 1,919 RGB pixels, but its physical row stride is padded. Kernels receive `pitch`, calculate `row = base + y * pitch`, and then access pixels. Assuming tightly packed rows corrupts indexing; ignoring alignment can reduce throughput for every row.

5. Diagrams / Mental Models

```
Base pointer: aligned
base + 0    -> aligned float4
base + 4    -> aligned float, misaligned float4
base + 16   -> aligned float4
```

```
2D pitched storage:
row 0: [logical pixels][padding]
row 1: [logical pixels][padding]  <- starts at aligned pitch
```

Concept	Question to ask
Type alignment	Is address a multiple of <code>alignof(T)</code> ?
Coalescing alignment	How many aligned sectors does the warp span?
Row pitch	Did indexing use the returned byte stride?
Struct layout	Do size, padding, and offsets match both sides?

6. Common Interview Questions

1. **What is alignment?** An address satisfying a type or transaction boundary. **Expected:** multiple-of-N definition. **Mistake:** confusing it with contiguous allocation.
2. **Why does alignment matter on GPUs?** It enables valid/effective wide accesses and fewer boundary-crossing transactions. **Expected:** correctness plus performance. **Mistake:** saying all misalignment crashes.
3. **Does `cudaMalloc` solve alignment?** It aligns the base, not arbitrary offsets. **Expected:** final-address reasoning. **Mistake:** assuming every subpointer is aligned.
4. **What is structure padding?** Unused bytes inserted to align members and the aggregate. **Expected:** `sizeof` may exceed field sum. **Mistake:** serializing raw structs without layout awareness.
5. **How do alignment and coalescing differ?** Alignment concerns boundaries/requirements; coalescing concerns warp addresses combining into transactions. **Expected:** related but distinct. **Mistake:** using them as synonyms.
6. **Why use pitched allocation?** To provide suitably aligned row starts and support 2D copies. **Expected:** use returned pitch. **Mistake:** indexing with logical width.
7. **When is `float4` useful?** When data and final pointers are aligned and four components are naturally processed together. **Expected:** instruction count/throughput nuance. **Mistake:** casting any float pointer.
8. **Can misalignment change correctness?** Violating language/type alignment requirements can be undefined or unsupported; some scalar accesses merely cost more. **Expected:** never rely on accidental behavior. **Mistake:** treating it only as performance.
9. **How would you verify struct layout?** `static_assert`, `sizeof`, `alignof`, and `offsetof` on relevant compilation targets. **Expected:** explicit layout/serialization when needed. **Mistake:** guessing from member order.
10. **How can an aligned warp access still use multiple transactions?** Its total byte range may cover several sectors; aligned does not mean one transaction. **Expected:** access width and warp span. **Mistake:** equating aligned with single-transaction.

7. Deep-Dive Questions

1. **Why can an offset of one float hurt a contiguous warp?** The range shifts across an extra aligned sector boundary, reducing useful-byte efficiency.

2. **What is over-alignment?** Requesting stricter alignment than the natural type; useful for vector instructions/layout but may increase padding.
3. **How should byte buffers be parsed safely?** Validate offsets/sizes and copy into an aligned object or use defined byte-wise mechanisms rather than unsafe misaligned casts.
4. **Why might vectorization not improve runtime?** Memory bandwidth may already dominate, the compiler may vectorize, or added register pressure/instruction handling may offset gains.
5. **How does alignment affect host transfers?** Page-locked allocation and transfer APIs have page-level concerns; device-side transaction alignment is a separate layer.

8. Comparison Tables

Aligned access	Misaligned/boundary-crossing access
Meets type requirements	May violate requirements
Enables natural-width instructions	May split/narrow operations
Minimizes extra sectors	Can touch an extra sector
Easier to vectorize	Unsafe to blindly vector-cast

9. Common Mistakes

- Checking allocation alignment but not subpointer offsets.
- Treating `sizeof(struct)` as the sum of fields.
- Casting to vector types without proving alignment.
- Ignoring returned pitch.
- Adding packing directives that create misaligned members.

10. Edge Cases / Special Cases

Zero-copy mapped host memory, external graphics buffers, custom allocators, and interop APIs may have different guarantees. Packed network/file formats should be decoded rather than directly reinterpreted. Arrays of over-aligned types need compatible allocation. Alignment requirements and transaction sizes should be taken from the language/API and target architecture, not folklore.

11. How to Explain in Interview

“Alignment means the final address satisfies the access type's byte boundary. Good alignment enables natural-width loads and avoids crossing unnecessary memory sectors; I check offsets, struct layout, and pitch, not just the allocation base.”

12. Quick Revision Notes

- `address % alignment == 0`.
- Base alignment does not guarantee offset alignment.
- Struct padding affects size and bandwidth.
- Pitch is a byte stride returned by the allocator.
- Trap: vector casts require proven alignment.

13. Practice Tasks

1. Print `sizeof`, `alignof`, and member offsets for several particle structs.
2. Benchmark aligned and one-float-offset vector reads.
3. Allocate a pitched 2D array and write correct row indexing.
4. Add compile-time checks for a host/device record layout.

14. Final Cheat Sheet

Item	Answer
Core definition	Address is a multiple of the required boundary
Why it matters	Correct wide accesses and fewer split transactions
Most asked	Offsets, padding, pitch, vector loads
Main comparison	Type alignment vs warp coalescing
One-line answer	"Check the final address and layout, not only the base allocation."

Cache Behavior

1. Overview

Definition. GPU caches retain recently accessed memory in fast on-chip storage so later accesses may avoid going to high-latency device memory. Typical CUDA GPUs have per-SM L1-related caching and a device-wide L2 cache; read-only/texture and constant paths provide additional behavior for suitable access patterns.

Caches matter because they can reduce latency and device-memory traffic, but their benefit depends on locality, working-set size, contention, and architecture. They are used implicitly by almost every kernel and deliberately in read-mostly lookup tables, textures, stencils, and repeated kernel pipelines. Interviewers ask about caches to see whether candidates understand locality without assuming CPU cache rules transfer unchanged to GPUs.

2. Core Idea

A cache is a small desk holding recently used pages from a large archive. **Temporal locality** means asking for the same page again soon. **Spatial locality** means asking for nearby content already brought with that page.

```
// Spatial locality across a warp
float x = a[base + threadIdx.x];

// Temporal locality inside a thread
float x = a[i];
use(x); use_again(x); // ideally retained in a register; otherwise cache may help
```

Step by step:

1. A warp requests memory sectors.
2. Cache tags are checked.
3. A hit returns data from cache; a miss requests the next level.
4. Fetched cache lines/sectors may evict older data.
5. Reuse helps only if it occurs before eviction and follows relevant visibility rules.

Registers and shared memory are explicit locality mechanisms; caches are mainly hardware managed. A strong answer considers both.

3. Important Subtopics

L1 and L2

L1 is close to an SM and optimized for local access; L2 is shared across the GPU and is important for global-memory traffic and inter-kernel reuse. Exact organization, policies, and capacities vary. Interview angle: do not promise that data remains cached.

Temporal locality

Reusing an address soon can produce hits. If reuse is predictable within a block, shared memory or registers may give more control than hoping the cache retains it.

Spatial locality

Fetching one region brings a granularity larger than a scalar. Adjacent accesses exploit the fetched data; scattered accesses waste it. This overlaps with coalescing but is not identical: coalescing combines one warp instruction's requests, while caching can serve reuse over time.

Read-only, texture, and constant access

Texture-oriented paths are useful for spatially local 2D access and specialized addressing/filtering through texture objects. Constant memory excels when a warp reads the same small read-only address because it can broadcast; divergent constant addresses may serialize.

Cache thrashing

A working set larger than effective capacity, unfortunate access patterns, or interference from other warps can evict useful lines before reuse. Tiling can shrink the active working set.

Memory consistency

A cached value is not automatically synchronized across threads. Atomics, barriers, fences, and kernel/stream ordering establish required visibility; cache presence alone does not.

4. Real-World Example

In graph analytics, neighbor lists are irregular and difficult to coalesce. Reordering vertices can cluster related adjacency lists, improving spatial locality and L2 hit rate. Frequently reused metadata may remain hot in L2, while the large edge array streams from memory. Profiling guides whether reordering is worth its preprocessing cost.

5. Diagrams / Mental Models

```

Thread registers
  |
Shared memory (explicit, per block)
  |
L1 / read-only paths (near an SM)
  |
L2 (shared across GPU)
  |
Device DRAM

```

Pattern	Cache outlook
Reuse soon	Good temporal locality
Adjacent addresses	Good spatial locality
One pass over huge array	Mostly streaming; limited reuse
Random accesses over huge set	Low hit rate likely
Uniform small read-only table	Constant/cache path may excel

6. Common Interview Questions

- What does a GPU cache do?** Serves recently/nearby accessed data faster and reduces DRAM traffic.
Expected: hierarchy and locality. **Mistake:** saying it guarantees low latency.
- L1 versus L2?** L1 is closer and generally SM-local; L2 is larger and shared device-wide. **Expected:** architecture varies. **Mistake:** quoting universal sizes/policies.
- Temporal versus spatial locality?** Reuse of the same data over time versus nearby addresses.
Expected: GPU example. **Mistake:** confusing spatial locality with synchronization.
- Coalescing versus caching?** Coalescing reduces sectors for a warp instruction; caching serves retained data across accesses. **Expected:** both can matter. **Mistake:** treating a cache hit as coalescing.
- When prefer shared memory?** When block-local reuse is predictable and explicit staging pays.
Expected: control and barriers/cost. **Mistake:** copying every load through shared memory.
- What is cache thrashing?** Useful lines are repeatedly evicted before reuse. **Expected:** capacity/conflict/interference. **Mistake:** assuming random access always means thrashing.
- Why can constant memory be fast?** Warp-uniform reads can broadcast from a small cached space.
Expected: divergent addresses may serialize. **Mistake:** using it for large writable arrays.
- How do you evaluate cache behavior?** Inspect hit rates, sectors, DRAM bytes, stalls, and runtime in a profiler. **Expected:** correlate metrics with code. **Mistake:** optimizing hit rate alone.
- Can one kernel benefit from a previous kernel's cache data?** Sometimes via L2 residency, but it is not a correctness guarantee and may be evicted. **Expected:** stream ordering for visibility. **Mistake:** relying on cache persistence.
- Does higher hit rate always mean faster?** No; workload, latency hiding, instruction count, and requested bytes matter. **Expected:** end-to-end measurement. **Mistake:** metric worship.

7. Deep-Dive Questions

1. **Why may a streaming kernel still perform well with low hit rate?** Coalesced requests can saturate DRAM bandwidth, and many warps hide latency; reuse is not required for efficient streaming.
2. **What is cache-line overfetch?** More bytes are fetched than used. It is harmful in sparse/strided access even if nominal bandwidth looks high.
3. **How can data layout improve caches?** SoA, blocking, reordering, and compression reduce footprint and place jointly used values together.
4. **How does L2 help host/device-visible operations?** It participates in device global-memory traffic and coherency mechanisms, but host visibility still requires API-defined synchronization.
5. **When is software-managed caching worse?** If hardware cache already captures reuse, explicit shared staging adds instructions, barriers, and resource pressure.

8. Comparison Tables

Hardware cache	Shared memory
Automatically managed	Explicitly loaded/indexed
Replacement not guaranteed	Contents controlled within block
No explicit load barrier	Requires safe cooperation/barriers
Good for irregular/incidental locality	Good for predictable block reuse

9. Common Mistakes

- Assuming CPU cache sizes and policies apply to GPUs.
- Treating cache residency as synchronization.
- Optimizing cache hit rate without checking bytes or runtime.
- Ignoring that registers may be the best place for thread-local reuse.
- Expecting caches to fully rescue scattered, one-use accesses.

10. Edge Cases / Special Cases

Atomics, peer memory, mapped host memory, managed memory, and external resources can follow specialized coherence and caching rules. Some GPUs allow L1/shared capacity preferences or L2 persistence hints, but these are tuning mechanisms, not portable guarantees. Very small constant tables with uniform access behave differently from divergent lookup tables.

11. How to Explain in Interview

"GPU caches reduce latency and DRAM traffic when accesses have temporal or spatial locality. L1 is closer to an SM and L2 is shared, but cache residence is opportunistic, so I use registers/shared memory for predictable reuse and profile hit rate, sectors, and DRAM traffic."

12. Quick Revision Notes

- Locality, working-set size, and eviction determine benefit.
- Coalescing and caching solve related but different problems.
- Shared memory is explicit; caches are hardware managed.
- Constant memory is strongest for warp-uniform small data.

- Trap: cache state never replaces synchronization.

13. Practice Tasks

1. Benchmark sequential, strided, and random reads over increasing array sizes.
2. Compare repeated global reads with register reuse and shared tiling.
3. Test a small uniform table through constant and global memory.
4. Profile L2 hit rate across two ordered kernels sharing data.

14. Final Cheat Sheet

Item	Answer
Core definition	Hardware retains recently accessed memory regions
Why it matters	Lower latency and less DRAM traffic
Most asked	L1/L2, locality, constant memory, thrashing
Main comparison	Cache vs explicitly managed shared memory
One-line answer	"Design for locality, but treat cache residence as an optimization—not a guarantee."

Pinned Host Memory

1. Overview

Definition. Pinned, or page-locked, host memory is RAM that the operating system will not page out. GPU DMA engines can transfer directly between it and device memory, and CUDA can use it for asynchronous copies and, when mapped, device access.

Pinned memory matters because pageable buffers normally require the runtime to stage data through a temporary pinned buffer. It is used in high-throughput data loaders, streaming pipelines, networking-to-GPU paths, and overlapped copies. Interviewers ask about it to test DMA, asynchronous execution, and the cost of pinning—not just transfer speed.

2. Core Idea

A delivery truck needs a fixed loading dock. Pageable memory is like inventory whose shelf location may change; the runtime first moves it to a fixed dock. Pinned memory is already at the dock.

```
Pageable host -> temporary pinned staging -> DMA -> GPU
Pinned host   -----> DMA -> GPU
```

Step by step:

1. Allocate with a page-locked API such as `cudaMallocHost/cudaHostAlloc`, or register an existing range where appropriate.

2. Fill or consume the host buffer.
3. Submit `cudaMemcpyAsync` in a stream.
4. DMA proceeds while the host thread and, with supported hardware/stream scheduling, GPU compute may continue.
5. Synchronize before reusing or freeing the buffer.

3. Important Subtopics

Pageable versus pinned

Pageable memory is the normal OS-managed allocation. Pinned memory avoids transfer staging and enables genuinely asynchronous host-device copy behavior under the API's conditions.

Allocation versus registration

Allocate new pinned buffers with CUDA APIs or register existing host ranges. Registration has setup cost and page-alignment/ownership considerations. Long-lived reusable buffers amortize the cost.

Asynchronous copies

Pinned memory is necessary but not sufficient for overlap. The GPU needs suitable copy engines, operations must be in appropriate non-serializing streams, and dependencies must permit concurrency.

Mapped/zero-copy memory

Mapped pinned memory can be addressed from the device on supported systems. This avoids an explicit copy but accesses host memory over an interconnect, usually with higher latency/lower bandwidth than device DRAM. It can suit small, one-pass, latency-sensitive data.

System impact

Pinned pages reduce memory available for paging and can harm whole-system performance if overused. Allocation/registration is more expensive than ordinary heap allocation.

4. Real-World Example

A deep-learning input pipeline maintains two or three reusable pinned batches. While the GPU processes batch `N`, an async copy sends batch `N+1`, and CPU workers prepare batch `N+2`. Events prevent a buffer from being refilled until its copy finishes. This hides transfer time without pinning the entire dataset.

5. Diagrams / Mental Models

```
time --->
copy stream: [H2D batch 1][H2D batch 2][H2D batch 3]
compute:      [kernel 1]   [kernel 2]   [kernel 3]
CPU prep:     [prep 2]     [prep 3]     [prep 4]
```

Property	Pageable host memory	Pinned host memory
OS may page it	Yes	No
Direct DMA source/target	Usually staged	Yes
Async-copy overlap	Restricted/not reliable	Supported when other conditions hold
Allocation cost	Low	Higher
Safe to allocate excessively	More forgiving	No; harms system memory management

6. Common Interview Questions

1. **What is pinned memory?** Host RAM locked against paging for stable DMA addresses. **Expected:** page-locked and transfer role. **Mistake:** calling it GPU memory.
2. **Why is it faster for transfers?** It avoids runtime staging through temporary pinned storage. **Expected:** DMA path. **Mistake:** claiming CPU accesses are always faster.
3. **Why is it needed for async copies?** DMA needs stable physical pages while the host call returns. **Expected:** buffer lifetime/synchronization. **Mistake:** assuming `_Async` guarantees overlap for pageable memory.
4. **What are its disadvantages?** Costly allocation and reduced pageable RAM; overuse hurts the system. **Expected:** pool/reuse buffers. **Mistake:** pinning all data.
5. **Does pinned memory guarantee copy-compute overlap?** No. Hardware, streams, dependencies, and transfer directions matter. **Expected:** concurrency conditions. **Mistake:** focusing only on allocation type.
6. **What is zero-copy memory?** Device accesses mapped host memory directly. **Expected:** avoids explicit copy but uses interconnect latency/bandwidth. **Mistake:** equating it with device DRAM speed.
7. **When is zero-copy useful?** Small or one-use data, integrated systems, or when copy overhead exceeds reuse benefit. **Expected:** access-count trade-off. **Mistake:** repeated random accesses from discrete GPU.
8. **How should pinned buffers be managed?** Allocate a bounded pool, reuse it, track async completion, and free with the matching API. **Expected:** lifetime. **Mistake:** per-batch allocate/free.
9. **Can an existing allocation be pinned?** It may be registered with a host-registration API under its constraints. **Expected:** setup cost. **Mistake:** registering arbitrary short-lived/invalid ranges.
10. **Pinned versus unified memory?** Pinned memory is host-resident and transfer-oriented; managed memory provides a unified address with runtime migration/coherence. **Expected:** distinct ownership models. **Mistake:** treating both as "no-copy."

7. Deep-Dive Questions

1. **Why can small copies see little benefit?** Launch/API overhead dominates the payload transfer; batching matters more.
2. **How do NUMA effects matter?** Pinned pages allocated on a CPU socket far from the GPU's PCIe root may reduce bandwidth; affinity and placement can matter.
3. **What is write-combined host memory?** A host allocation mode optimized for CPU writes and device reads; CPU reads may be slow, so it suits one-way staging.
4. **What synchronization bug is common?** Reusing or freeing a pinned buffer before the async transfer completes.

5. **Why pool pinned memory?** Allocation and OS pin/unpin operations are expensive; pooling amortizes them and bounds system pressure.

8. Comparison Tables

Pinned transfer buffer	Mapped zero-copy buffer
Explicit H2D/D2H copy	Device directly addresses host storage
Data can then reside in fast device DRAM	Every access traverses host/interconnect path
Best for reuse on GPU	Best for limited/one-pass access
Can overlap via streams	Avoids copy but not access cost

9. Common Mistakes

- Pinning the whole dataset.
- Allocating pinned memory in the hot loop.
- Reusing a buffer before its stream/event completes.
- Assuming pinned means physically on the GPU.
- Expecting async overlap in the default/serialized dependency chain.

10. Edge Cases / Special Cases

Integrated GPUs may share physical memory and change the zero-copy trade-off. OS pinning limits and container/runtime policies may constrain registration. Unified virtual addressing makes pointer values easier to manage but does not make host pages equivalent to device DRAM. Bidirectional engines and concurrent-copy capabilities differ by device.

11. How to Explain in Interview

“Pinned memory is page-locked host RAM that DMA can use directly. It enables efficient asynchronous transfers, but pinning is expensive and consumes a scarce system resource, so I use a small reusable buffer pool and synchronize its lifetime.”

12. Quick Revision Notes

- Pinned = host-resident, page-locked.
- Avoids pageable staging and enables async DMA.
- Overlap also needs streams, hardware, and independent work.
- Pool and reuse; do not pin everything.
- Trap: zero-copy avoids the copy, not interconnect latency.

13. Practice Tasks

1. Compare pageable and pinned bandwidth for several transfer sizes.
2. Build a double-buffered copy/compute pipeline with two streams.
3. Demonstrate the incorrect result/race from early buffer reuse, then fix it with events.
4. Compare explicit copy against mapped access for one-use and repeated-use data.

14. Final Cheat Sheet

Item	Answer
Core definition	Host RAM locked for stable DMA access
Why it matters	Faster staging and asynchronous transfers
Most asked	Pageable vs pinned, overlap, zero-copy, cost
Main comparison	Explicit pinned copy vs mapped host access
One-line answer	"Use a bounded pinned pool to feed DMA; do not pin the world."

Unified Memory

1. Overview

Definition. CUDA Unified Memory allocates managed data accessible through one virtual address from CPUs and supported GPUs. The runtime and driver maintain accessibility and coherence, often by migrating memory pages or mapping them across processors.

Unified Memory matters because it simplifies pointer management and enables data larger than GPU memory, but implicit page movement can cause stalls and unpredictable performance. It is used in rapid development, irregular structures, multi-GPU programs, and oversubscription. Interviewers ask about it to separate programming convenience from physical placement and performance.

2. Core Idea

A managed pointer is one library card valid at several branches; it does not mean the book is simultaneously on every desk. When another processor needs a page, the system may move or map it.

```
float* p;
cudaMallocManaged(&p, n * sizeof(float));
initialize_on_cpu(p);
kernel<<<grid, block>>>(p, n);
cudaDeviceSynchronize();
consume_on_cpu(p);
cudaFree(p);
```

Step by step:

1. Allocate a managed virtual range.
2. CPU touches pages, establishing initial residency.
3. GPU accesses pages; page faults or migration may occur.
4. Kernel continues after required pages become accessible.
5. CPU access after proper synchronization may migrate/map pages back.

Correct synchronization is still required; a unified pointer does not permit unsynchronized concurrent data races.

3. Important Subtopics

Virtual address versus physical residency

The address can remain the same while the page resides in host memory, GPU memory, or another supported location. Interview angle: unified address space is not uniform performance.

Demand paging and page faults

First touch on a processor can fault and trigger migration. Many small fault-driven migrations can stall kernels and underuse the interconnect.

Prefetching

`cudaMemPrefetchAsync` can move managed ranges toward a processor before use, converting reactive faults into planned asynchronous work. It needs correct stream ordering and a known phase boundary.

Memory advice

Advice can express preferred location, read-mostly use, or access expectations. It is a hint and must be validated on the target system.

Oversubscription

Managed allocations can exceed device-memory capacity on systems supporting migration. Performance depends on the working set; repeated eviction and refaulting causes thrashing.

Multi-GPU behavior

Pages may migrate, replicate for read-mostly use, or be remotely accessed depending on topology and platform. Correctness is easier than optimal placement.

4. Real-World Example

An irregular graph algorithm follows pointer-rich structures whose active region changes by iteration. Managed memory avoids manually flattening and copying every reachable object. Before each GPU phase, the program prefetches the expected frontier and adjacency ranges. If the frontier repeatedly exceeds device capacity, profiling may show migration thrashing and motivate partitioning.

5. Diagrams / Mental Models

```
Same managed virtual pointer p
CPU phase:  pages near CPU
            | prefetch / fault-driven migration
GPU phase:  pages in GPU memory
```

| synchronization + migration/mapping

CPU phase: accessible to CPU again

Managed-memory behavior	Performance implication
One large phase per processor	Prefetch can work well
CPU/GPU alternate on same pages often	Page ping-pong
Read-mostly shared data	Replication/advice may help
Working set exceeds GPU memory	Possible oversubscription and thrashing

6. Common Interview Questions

- What is Unified Memory?** A managed allocation accessible from CPU/GPU through one virtual address. **Expected:** runtime placement/coherence. **Mistake:** saying one physical copy is always shared.
- Does it eliminate data movement?** No; movement may become implicit through migration. **Expected:** faults/prefetch. **Mistake:** calling it zero-copy.
- What causes a managed-memory page fault?** A processor accesses a page not currently accessible/resident as needed. **Expected:** migration or mapping. **Mistake:** treating every fault as an application crash.
- How improve predictable phases?** Prefetch data to the next processor and order it in a stream. **Expected:** phase boundaries. **Mistake:** prefetching after the kernel starts.
- What is oversubscription?** Managed working data exceeds device memory. **Expected:** allowed on supported systems but may thrash. **Mistake:** assuming capacity becomes unlimited at full speed.
- Is synchronization still needed?** Yes, for operation completion and legal visibility/access. **Expected:** unified address does not remove races. **Mistake:** CPU reads immediately after launch.
- Unified versus explicit copies?** Managed is simpler/dynamic; explicit copies give predictable placement and scheduling. **Expected:** workload-dependent choice. **Mistake:** one universally wins.
- What is page thrashing?** Pages repeatedly migrate/evict because competing processors or oversized working sets alternate access. **Expected:** partition/restructure/prefetch. **Mistake:** adding more random prefetches.
- When is Unified Memory a good fit?** Complex/irregular data, prototyping, changing working sets, oversubscription, or when measured performance is adequate. **Expected:** convenience-performance trade-off. **Mistake:** restricting it to beginners.
- How do you profile it?** Inspect migration bytes, GPU page faults, fault groups, transfer timeline, and kernel stalls. **Expected:** correlate with access phases. **Mistake:** timing kernels without migration costs.

7. Deep-Dive Questions

- Why can first kernel execution be slow?** First-touch faults migrate pages during execution; warm runs may not represent cold behavior.
- How can CPU initialization be improved?** Initialize then prefetch the whole upcoming GPU range asynchronously before compute, or initialize on GPU when suitable.
- What does read-mostly advice enable?** The system may replicate read-only pages rather than migrate exclusive ownership, depending on platform support.

4. **Why does access granularity matter?** Migration occurs in pages/groups, so touching one byte may move substantially more data.
5. **How would you fix CPU-GPU ping-pong?** Separate ownership phases, partition data, batch updates, use distinct buffers, or keep frequently shared control data in a more suitable mechanism.

8. Comparison Tables

Unified Memory	Explicit device memory
One managed pointer	Separate host/device allocations
Placement may migrate implicitly	Programmer schedules copies
Easier irregular data/lifetime	More predictable transfer timing
Can oversubscribe on supported systems	Usually bounded by allocation capacity
Fault/thrashing risk	Copy/ownership bookkeeping burden

9. Common Mistakes

- Calling managed memory “no-copy.”
- Timing only the kernel after pages are already resident.
- Alternating CPU/GPU writes to the same pages.
- Omitting synchronization before host access.
- Assuming prefetch/advice are guarantees.
- Using oversubscription without checking the active working set.

10. Edge Cases / Special Cases

Behavior differs across operating systems, GPU capabilities, interconnects, and multi-GPU topologies. Concurrent managed access capabilities determine which access patterns are supported efficiently. Small control data may behave differently from bulk arrays. Memory allocated before device selection or accessed by unsupported peers can receive different placement treatment.

11. How to Explain in Interview

“Unified Memory gives CPU and GPU one managed virtual pointer, while the runtime migrates or maps pages to make them accessible. It simplifies ownership, but I still synchronize and manage locality with phased access, prefetching, and profiling to avoid page faults and thrashing.”

12. Quick Revision Notes

- Unified address does not mean uniform physical memory.
- Demand faults can migrate pages during kernels.
- Prefetch predictable working sets.
- Oversubscription works only if locality avoids thrashing.
- Trap: managed pointers do not remove synchronization or races.

13. Practice Tasks

1. Run a managed vector kernel cold and warm; compare timings.
2. Add prefetch before GPU and CPU phases.
3. Create alternating CPU/GPU access and observe page migration.
4. Process an allocation larger than device memory in chunks and compare locality orders.

14. Final Cheat Sheet

Item	Answer
Core definition	One managed virtual address with runtime placement/coherence
Why it matters	Easier ownership and possible oversubscription
Most asked	Page faults, prefetch, thrashing, synchronization
Main comparison	Managed migration vs explicit copies
One-line answer	"Unified Memory simplifies addresses, not the cost of moving data."

Memory Transfers CPU ↔ GPU

1. Overview

Definition. CPU↔GPU transfers move data between host memory and device-accessible memory, commonly through PCIe or a higher-bandwidth coherent interconnect. CUDA exposes synchronous/asynchronous copies, 2D/3D copies, peer copies, and transfer-related stream primitives.

Transfers matter because interconnect bandwidth and latency are usually far worse than on-device memory bandwidth. A fast kernel can lose end-to-end if input/output movement dominates. Transfers appear in every discrete-GPU application: data loading, inference, simulation, rendering, databases, and analytics. Interviewers ask about them to test end-to-end thinking and concurrency.

2. Core Idea

The GPU is a factory across a bridge. Sending one screw per truck is inefficient; send batches, keep materials at the factory, and overlap delivery with production.

Total time is approximately:

$$T_{\text{total}} = T_{\text{H2D}} + T_{\text{kernel}} + T_{\text{D2H}} + \text{synchronization/launch overhead}$$

With safe pipelining, elapsed time can approach the longest pipeline stage rather than their sum.

```
cudaMemcpyAsync(d_in, h_in, bytes, cudaMemcpyHostToDevice, stream);
kernel<<<grid, block, 0, stream>>>(d_in, d_out);
cudaMemcpyAsync(h_out, d_out, bytes, cudaMemcpyDeviceToHost, stream);
```

Operations in one stream are ordered. Multiple streams can overlap independent chunks when hardware and dependencies allow it.

3. Important Subtopics

Bandwidth and latency

Large copies approach interconnect bandwidth; tiny copies are dominated by fixed API/transaction overhead. Batch small objects into contiguous transfers.

Synchronous versus asynchronous APIs

A synchronous copy blocks relevant host progress until completion according to API semantics. Async submission returns earlier, but actual concurrency requires pinned host memory, suitable streams/hardware, and no blocking dependency.

H2D and D2H

Directions can have different concurrency characteristics. Some devices have engines capable of simultaneous transfers in opposite directions; query capability rather than assume.

Streams and overlap

Chunk input into buffers and pipeline H2D, kernel, and D2H. Events express cross-stream dependencies without synchronizing the whole device.

2D/3D transfers

Use pitch-aware APIs for images/volumes. They copy logical widths across rows/slices whose physical strides may differ.

Peer-to-peer and collectives

GPU-to-GPU transfers may avoid staging through host memory when topology and peer access support it. Distributed training often uses optimized collective libraries rather than manual CPU round trips.

Effective bandwidth

For a copy, $\text{bytes} / \text{time}$; for a kernel, count relevant reads and writes. Always time with GPU events or end-to-end wall time at the correct synchronization boundary.

4. Real-World Example

A video analytics server decodes frames on the CPU and runs GPU inference. It uses a ring of pinned host buffers and device buffers. One frame transfers while the previous frame runs inference and an earlier result transfers back. Only compact detections—not full intermediate tensors—return to the CPU.

5. Diagrams / Mental Models

Sequential: [H2D 0][K 0][D2H 0][H2D 1][K 1][D2H 1]

Pipelined:

H2D: [0][1][2][3]

Kernel: [0][1][2][3]

D2H: [0][1][2][3]

Optimization	What it reduces
Batch small copies	Fixed per-copy overhead
Pinned buffers	Pageable staging; enables async DMA
Multiple streams	Exposes copy/compute concurrency
Keep data on GPU	Transfer bytes and synchronization
Compression	Bytes, at cost of encode/decode compute

6. Common Interview Questions

1. **Why are transfers expensive?** The interconnect has higher latency and usually lower bandwidth than GPU-local memory. **Expected:** end-to-end impact. **Mistake:** blaming only API overhead.
2. **How reduce transfer cost?** Move fewer bytes, batch, pin, overlap, and retain data on device. **Expected:** prioritize elimination before acceleration. **Mistake:** streams as the only answer.
3. **What is an async copy?** A stream-enqueued transfer that can return control before completion. **Expected:** lifetime and dependency rules. **Mistake:** assuming it always overlaps.
4. **What enables copy-compute overlap?** Pinned host memory, capable engines, independent operations, suitable streams, and correct ordering. **Expected:** all conditions. **Mistake:** putting dependent work in a serial stream and expecting overlap.
5. **Why batch small transfers?** Fixed submission/setup latency is amortized over more bytes. **Expected:** packing cost trade-off. **Mistake:** copying many fields separately.
6. **How measure transfer time?** GPU events around stream work or wall-clock timing with explicit completion, depending on question. **Expected:** synchronize correctly. **Mistake:** timing only enqueue duration.
7. **What is bidirectional overlap?** Concurrent H2D and D2H on devices with suitable copy engines. **Expected:** device capability. **Mistake:** guarantee on all GPUs.
8. **How handle matrices with pitch?** Use 2D copy widths in bytes and source/destination pitch. **Expected:** logical versus physical row stride. **Mistake:** flat copy of padded storage.
9. **When can GPU compute justify a transfer?** When saved CPU time or accelerated work exceeds movement/launch cost, preferably over a pipeline. **Expected:** end-to-end break-even. **Mistake:** comparing kernel time alone.
10. **Why can `cudaDeviceSynchronize()` hurt?** It stalls the host and all device work, destroying overlap. **Expected:** use stream/event-scoped dependencies. **Mistake:** removing synchronization required for correctness.

7. Deep-Dive Questions

1. **How choose chunk size?** Large enough to amortize overhead and saturate bandwidth, small enough to pipeline and fit buffers; benchmark a range.
2. **Why might multiple streams show no overlap?** Same engine contention, default-stream semantics, pageable memory, dependency serialization, too-small work, or hardware limits.
3. **Can compression help?** Yes when reduced transfer time exceeds compression/decompression cost and latency; GPU-side codecs may keep the pipeline on device.
4. **What is the break-even equation for offload?** Offload wins when $T_{H2D} + T_{GPU} + T_{D2H} + \text{overhead} < T_{CPU}$, including queueing and synchronization.
5. **How does topology affect multi-GPU transfer?** PCIe switches, NUMA placement, and high-speed GPU links determine whether peer paths are direct and their bandwidth.

8. Comparison Tables

Synchronous transfer	Asynchronous transfer
Simpler control flow	Enables pipelining
Host waits according to call semantics	Host can enqueue more work
Easy lifetime management	Buffer cannot be reused early
Often serializes stages	Overlap depends on hardware/dependencies

9. Common Mistakes

- Timing enqueues without waiting for completion.
- Making many tiny transfers.
- Calling device-wide synchronization between every stage.
- Assuming streams automatically create independence.
- Ignoring host buffer lifetime and pinning.
- Returning full intermediate arrays when only a summary is needed.

10. Edge Cases / Special Cases

For integrated GPUs, physical-memory sharing changes the copy model. Small messages may favor direct mapped access. Transfers involving pageable memory may have partial or implementation-dependent asynchronous behavior. The legacy default stream can introduce implicit synchronization depending on compilation/runtime configuration. Error reporting can surface at later synchronization calls, so check both launch and completion errors.

11. How to Explain in Interview

“CPU–GPU transfers cross a slower interconnect, so I optimize end-to-end time by first eliminating bytes, then batching and using pinned buffers, and finally pipelining independent H2D, compute, and D2H stages with streams and events.”

12. Quick Revision Notes

- Interconnect bandwidth is far below device-local bandwidth.
- Batch tiny copies; reuse bounded pinned buffers.

- Async does not automatically mean overlap.
- Use events/streams instead of global synchronization where possible.
- Trap: always include movement when judging GPU speedup.

13. Practice Tasks

1. Plot bandwidth versus transfer size for pageable and pinned buffers.
2. Implement sequential and double-buffered vector processing.
3. Use events to measure each pipeline stage.
4. Copy a pitched image correctly with a 2D API.
5. Calculate the CPU/GPU offload break-even point from measured numbers.

14. Final Cheat Sheet

Item	Answer
Core definition	Movement across the host-device memory/interconnect boundary
Why it matters	Often dominates otherwise fast kernels
Most asked	Pinned memory, async copies, overlap, batching
Main comparison	Sequential vs pipelined transfers
One-line answer	"Move less, batch what remains, and overlap only independent work."

Avoiding Unnecessary Transfers

1. Overview

Definition. Avoiding unnecessary transfers means designing the application so data crosses the CPU–GPU boundary only when another processor truly needs it, and only in the smallest useful representation.

This is often the highest-value memory optimization: a transfer eliminated costs zero bandwidth and needs no overlap machinery. It is used in persistent GPU pipelines, fused preprocessing/inference/postprocessing, GPU databases, simulations, rendering, and batched services. Interviewers ask about it because strong candidates optimize the whole dataflow rather than an isolated kernel.

2. Core Idea

Do not repeatedly carry ingredients between two kitchens. Decide where the meal is prepared, keep intermediate ingredients there, and carry back only the plated result.

Bad flow:

```
CPU -> GPU preprocess -> CPU -> GPU inference -> CPU -> GPU postprocess -> CPU
```

Better flow:

```
CPU -> GPU preprocess -> inference -> postprocess -> compact result -> CPU
```

Step by step:

1. Draw every buffer and which processor produces/consumes it.
2. Mark required external inputs and final outputs.
3. Keep intermediate data where the next consumer runs.
4. Fuse or chain device operations without host round trips.
5. Transfer only changed ranges or compact summaries.
6. Remove synchronization used solely to let the CPU inspect intermediate state.

3. Important Subtopics

Data residency and ownership

Allocate long-lived device buffers once and reuse them across iterations. Make one side authoritative for each phase so accidental mirror copies are not needed.

Kernel fusion

Combining adjacent element-wise stages can keep intermediate values in registers/shared memory and avoid global-memory traffic as well as host transfers. Fusion may increase registers or duplicate work, so stop when it reduces scheduling flexibility or occupancy materially.

Device-side reductions and filtering

Reduce, aggregate, compact, or select on the GPU, then return a small answer instead of the full dataset. Example: return 100 detections rather than a full probability tensor.

Dirty ranges and incremental updates

Copy only changed elements, tiles, or parameters. Track versions or dirty regions when update sparsity justifies the bookkeeping.

Recompute versus transfer

Cheap deterministic values may be recomputed on the GPU faster than transferred. Examples include indices, normalization factors, and simple derived fields.

Device-side control flow and libraries

Use GPU libraries and device-side primitives so intermediate results remain device-resident. CUDA Graphs can reduce repeated launch overhead, although they do not themselves eliminate required data movement.

Lazy materialization

Do not copy results to the CPU "just in case." Materialize a host representation only when a host consumer requests it.

4. Real-World Example

A GPU database scans a column, applies a predicate, aggregates matching rows, and returns a few grouped totals. Copying the entire column or match bitmap to the CPU would dominate. Keeping scan, filter, and reduction on device reduces the output from gigabytes to kilobytes and removes intermediate synchronization.

5. Diagrams / Mental Models

Before:

```
Host A --copy--> Device A --kernel--> Device B --copy--> Host B
Host B --copy--> Device B --kernel--> Device C --copy--> Host C
```

After:

```
Host A --one copy--> Device A -> Device B -> Device C
                                --small final copy--> Host result
```

Question	Preferred action
Does CPU consume this intermediate?	If no, keep it on GPU
Does GPU need the whole host object?	Pack/copy only used fields
Did only a small range change?	Update that range
Is derived data cheap?	Recompute on device
Is output huge but decision small?	Reduce/filter before D2H

6. Common Interview Questions

- What is the best way to optimize a transfer?** Eliminate it if no real consumer requires it. **Expected:** dataflow first. **Mistake:** immediately proposing pinned memory.
- How keep data resident?** Reuse device allocations and chain kernels/libraries on the same device data. **Expected:** lifetime/ownership plan. **Mistake:** allocating and copying every iteration.
- What is kernel fusion?** Combining stages so intermediates need not be materialized or transferred. **Expected:** global-memory/register trade-off. **Mistake:** fusing every kernel.
- How reduce D2H output?** Perform reduction, filtering, compaction, encoding, or top-k on GPU. **Expected:** transfer final information only. **Mistake:** returning all data for CPU postprocessing.
- When copy only dirty ranges?** When updates are sparse and tracking cost is below full-copy cost. **Expected:** range batching. **Mistake:** thousands of tiny copies.
- Can recomputation be faster than transfer?** Yes, for cheap derived values because GPU arithmetic is abundant relative to interconnect bandwidth. **Expected:** measure compute/byte trade-off. **Mistake:** recomputing expensive or nondeterministic state.
- Why avoid host inspection between kernels?** It forces D2H transfer and synchronization, breaking the pipeline. **Expected:** device-side decision or deferred diagnostics. **Mistake:** removing required correctness checks silently.
- How do libraries help residency?** GPU library outputs can feed later GPU operations directly. **Expected:** compatible layouts/streams. **Mistake:** converting through host formats between libraries.

9. **What if CPU and GPU both need data?** Define phases, immutable snapshots, managed/pinned strategies, or compact deltas; synchronize ownership explicitly. **Expected:** correctness first. **Mistake:** unsynchronized shared access.
10. **How prove a transfer is unnecessary?** Trace producers/consumers and show no host/device consumer needs that representation at that point. **Expected:** end-to-end profile and dataflow. **Mistake:** deleting copies without checking dependencies.

7. Deep-Dive Questions

1. **When can fusion hurt?** It can raise register pressure, reduce occupancy, duplicate shared inputs, enlarge code, or remove concurrency between stages.
2. **How would you handle a tiny CPU control decision based on GPU data?** Produce a compact flag/scalar on device, copy only it asynchronously, or restructure control to remain on device if profitable.
3. **How do you avoid transferring unchanged model weights?** Upload once, retain device ownership across requests, and version/update only changed parameters.
4. **Can Unified Memory eliminate unnecessary transfers?** It removes explicit copy code, not physical movement. Bad alternating access can create more migration than an explicit design.
5. **How does batching reduce transfers?** It combines payloads, amortizes fixed overhead, improves contiguous layout, and can make one device-resident pipeline serve many requests.

8. Comparison Tables

Transfer optimization	Benefit	Cost/risk
Eliminate transfer	Removes all bytes and overhead	Requires dataflow redesign
Reduce/compact first	Much smaller output	Extra GPU compute
Batch transfers	Amortizes latency	Adds batching delay/buffering
Overlap transfer	Hides some elapsed time	Complexity; bytes still move
Unified Memory	Simpler code	Movement becomes implicit/unpredictable

9. Common Mistakes

- Optimizing copy bandwidth before questioning the copy.
- Copying intermediates for logging in production hot paths.
- Reallocating and re-uploading constant data per request.
- Fusing so aggressively that register spills erase the gain.
- Tracking individual dirty elements and issuing thousands of tiny copies.
- Treating Unified Memory as transfer elimination.

10. Edge Cases / Special Cases

Correctness, observability, and recovery may require some host-visible state. In latency-sensitive applications, batching can violate response targets. If the next stage runs on CPU, leaving data on GPU merely postpones a necessary copy. Multi-GPU pipelines need topology-aware residency; keeping data “on GPU” is not enough if every stage moves it across a slow peer path.

11. How to Explain in Interview

"I first draw the producer-consumer dataflow and remove host round trips for intermediates. I keep long-lived data on the GPU, chain device operations, reduce or compact results before copying back, and transfer only changed ranges. Pinned memory and overlap come after byte elimination."

12. Quick Revision Notes

- The cheapest transfer is the one not performed.
- Keep intermediate and constant data device-resident.
- Reduce/filter/compact before D2H.
- Recompute cheap derived values when cheaper than movement.
- Trap: overlap hides time but does not reduce traffic.

13. Practice Tasks

1. Draw the dataflow of a three-stage CPU/GPU application and mark avoidable round trips.
2. Fuse two element-wise kernels and compare transfer/global-memory traffic and register use.
3. Replace a full-array D2H copy with a device reduction returning one scalar.
4. Implement versioned parameter updates that copy only changed ranges.
5. Compare transfer, recomputation, and managed-memory migration for a derived array.

14. Final Cheat Sheet

Item	Answer
Core definition	Cross the host-device boundary only for true consumers
Why it matters	Eliminates bandwidth, latency, and synchronization cost
Most asked	Residency, fusion, compaction, recompute vs copy
Main comparison	Eliminate vs reduce vs overlap transfers
One-line answer	"Keep the pipeline on the GPU and return only the information the CPU needs."

Cross-Topic Interview Summary

Symptom	First concepts to inspect	Typical fix
High DRAM bytes, low useful work	Coalescing, layout, cache locality	Unit-stride mapping, SoA, blocking
Repeated global reads	Tiling, registers, caches	Reuse in registers/shared memory
Slow shared-memory instruction	Bank conflicts	Change stride or pad tile
Low residency or local loads	Register usage	Reduce live ranges/unrolling; verify spills
Split or unsafe wide accesses	Alignment	Fix final offsets, pitch, and layout

Symptom	First concepts to inspect	Typical fix
Kernel fast, application slow	Host-device transfers	Eliminate, batch, pin, pipeline
Managed kernel stalls	Unified-memory faults	Prefetch, phase ownership, reduce working set

A practical optimization order

1. Measure end-to-end time, including transfers and synchronization.
2. Remove unnecessary CPU↔GPU movement.
3. Make remaining transfers large, pinned, and pipeline-friendly.
4. Fix global-memory coalescing and alignment.
5. Increase useful locality through registers, caches, or tiling.
6. Remove measured bank conflicts and spills.
7. Recheck occupancy, bandwidth, stalls, and correctness after every change.

The central interview principle is simple: **optimize useful data movement through the entire hierarchy, not merely instruction count inside one kernel.**